PROJECT GUIDE · WORKING · TECH STACK · INTERVIEW PREP

Football Analytics Platform

An end-to-end data-engineering, data-science and analytics platform for men's international tournament football: World Cup, Euro, Copa América and Nations League, with a broadcast-style React dashboard.

Live demo: football-analytics-xi.vercel.app

Source code: github.com/alirazach476/football-analytics

Author: Ali Raza

1. The project in one page

What it does. It collects open football data from six sources, cleans and unifies it through a Bronze → Silver → warehouse → Gold pipeline, analyses it with statistics and machine learning, and serves it through a FastAPI backend to a React dashboard where every tournament edition has its own visual identity built from its logo.

1,830 matches (1,779 played + 51 coming soon)	**103** national teams	**8,410** canonical players after entity resolution	**1.03 M** event rows (passes, shots, goals…)
30 tournament editions	**85** interactive chart types	**115** dbt models and tests	**223 + 6** Python + front-end tests

Golden rules

- **Nothing is invented.** Missing data stays empty (NULL) and is reported; a missing value is never shown as zero.
- **Provenance is kept** for every record (source, version, hash); raw collected data is never modified or deleted.
- **Upcoming fixtures are kept and shown as "Coming soon"**, but never enter statistics or model targets.
- **Predictions are always labelled** as predictions, and weak model results are reported plainly.
- **Scope**: four men's competitions only (the UEFA Champions League is excluded by design). Only openly licensed sources are used; FIFA, UEFA and CONMEBOL sites are not scraped because their terms forbid it.

2. How it works, end to end

Sources → Bronze → Silver → PostgreSQL star schema → dbt Gold → ML + statistics → FastAPI → React on Vercel

2.1 Sources and Bronze (ingestion)

Source	Gives	Licence
OpenFootball	fixtures, results, goal scorers (Euro, Copa, World Cup 2026)	CC0
martj42 international results	results, goal scorers, a few assists	CC0
Fjelstul World Cup database	World Cup matches, goals, squads	CC BY-SA 4.0
StatsBomb Open Data	event-level data, line-ups, managers, shots, passes	attribution terms
Wyscout public dataset	event data for some editions	CC BY 4.0
Wikipedia (MediaWiki API)	match boxes (line-ups, cards, managers), awards, venues, World Cup 2030 facts	CC BY-SA 4.0

Every downloaded file is registered with a **SHA-256 hash** and a version, and frozen snapshots are kept, so any result can be traced back to the exact bytes it came from. Files from the excluded competition are flagged and never processed.

2.2 Silver (clean, standardise, resolve)

- One schema for all sources: matches, teams, players, line-ups, events, venues, squads. Original raw values are kept next to the standardised ones.
- **Entity resolution** merges the same person across sources (8,410 canonical players): same name and birth date, or name containment plus same national team and never in one line-up together. Ambiguous cases go to a review file instead of being guessed.

- **Pandera** validates the Silver tables (13 checks); source disagreements are written to a conflicts report, and Wikipedia only fills nulls, it never overwrites a licensed dataset.

2.3 Warehouse and Gold

- **PostgreSQL star schema** (`wh`): dimensions (competition, season, team, player, venue, referee, date, stage) and facts (matches, team/player match stats, appearances, events, standings, awards, managers) with surrogate keys and constraints.
- **dbt** builds staging, intermediate and 11 Gold marts, with 115 models and tests.
- **Monitoring schema**: pipeline runs, step attempts, data-quality results, source coverage, freshness, ingestion history. The dashboard's data-engineering pages read these tables.
- **Orchestration**: Dagster assets or a plain runner; steps are idempotent (skipped when input hashes are unchanged) and retried, with every attempt logged.

2.4 Statistics and machine learning

- **Statistics**: ten statistical questions (details in `docs/STATISTICAL_REPORT.md`) answered with hypothesis tests, confidence intervals and permutation methods; goal counts are discrete and over-dispersed, so rank-based and count-model tools are used.
- **Features** are built *only from matches before each match date* (Elo-style strength, recent form, tournament experience), so there is no leakage; a chronological train / validation / test split is used.
- **Models**: prior baseline, one-feature Elo logistic baseline, logistic regression, random forest and XGBoost; selection on validation log loss; bootstrap confidence intervals on the test set; permutation importance, SHAP and partial dependence; calibration curves; goal-count models (Poisson-family); K-Means and hierarchical clustering of teams and players; MLflow tracking.

> **Honest result.** On 275 test matches, accuracy is about 55–57 %. XGBoost (log loss 0.956) does **not** clearly beat the one-feature Elo baseline (0.942); the confidence intervals overlap. Goal totals barely beat the base rate and team clusters are weak. The project states this plainly instead of overselling.

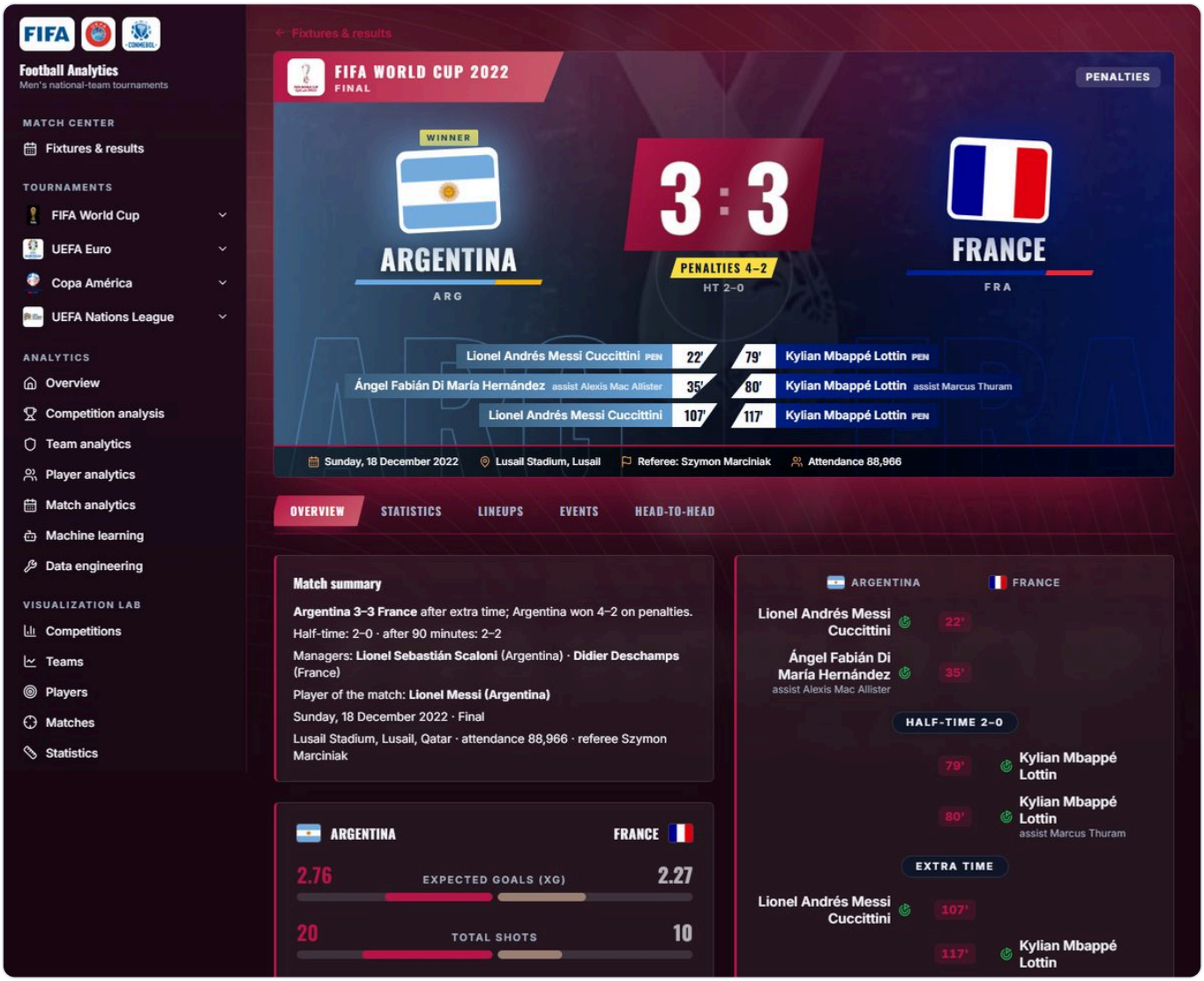
2.5 API and dashboard

- **FastAPI** exposes a registry of 85 chart ids, whitelisted queries, tournament pages, the Match Center, fixtures, standings, exports and the match predictor. Python builds each Plotly figure and returns it as JSON; React renders it. One code path serves analysis and UI, and every "not shown: reason" rule is preserved.
- **React + TypeScript** (Vite, Tailwind, TanStack Query, framer-motion): per-edition themes, broadcast scoreboard, line-ups on a pitch, fixtures with filters, a 7-page analytics section and a 7-page visualization lab with cross-filtering, table view and CSV / HTML / PNG export on every chart.

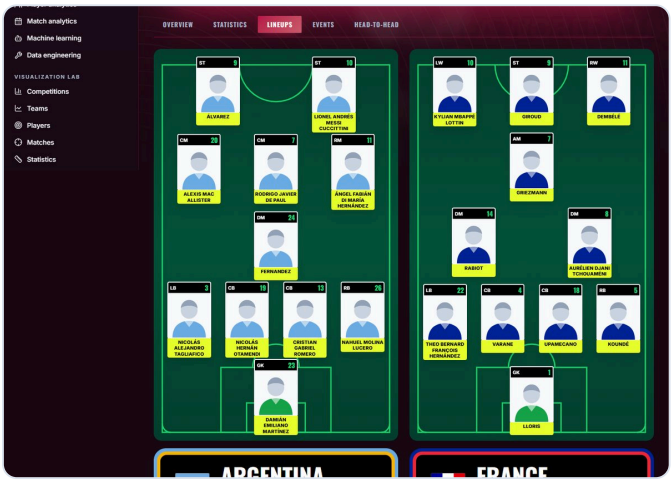
3. Tech stack and why

Area	Choice	Why
Language	Python 3.12, TypeScript	One language across ingestion, analysis and API; TypeScript for a safe front end
Storage	Parquet (Bronze/Silver), PostgreSQL (warehouse)	Cheap immutable files for raw layers; relational constraints and SQL for serving
Transform	dbt	Versioned SQL marts with built-in tests and lineage
Validation	Pandera + dbt tests + constraints	Three layers of defence, each catching different errors
Orchestration	Dagster / runner, monitoring tables	Visible runs, retries, idempotency, freshness
ML	scikit-learn, XGBoost, SHAP, MLflow	Standard, explainable, reproducible; a baseline keeps claims honest
API	FastAPI + psycopg 3	Typed, fast, automatic docs; reuses the existing Python chart code
Charts	Plotly (server-side JSON) + Matplotlib/Seaborn twins	Interactive charts plus static exports without rewriting charts in JS
Front end	React 18, Vite, Tailwind, TanStack Query, framer-motion, flag-icons	Fast builds, cached queries, themeable design tokens, animation
Hosting	Vercel, Neon, Cloudflare tunnel, Docker	Free tier friendly; see section 6

4. Dashboard tour



Match Center: broadcast scoreboard in each nation's colours, scorers with assists, managers, comparison statistics.



Line-ups: formation derived from recorded positions.



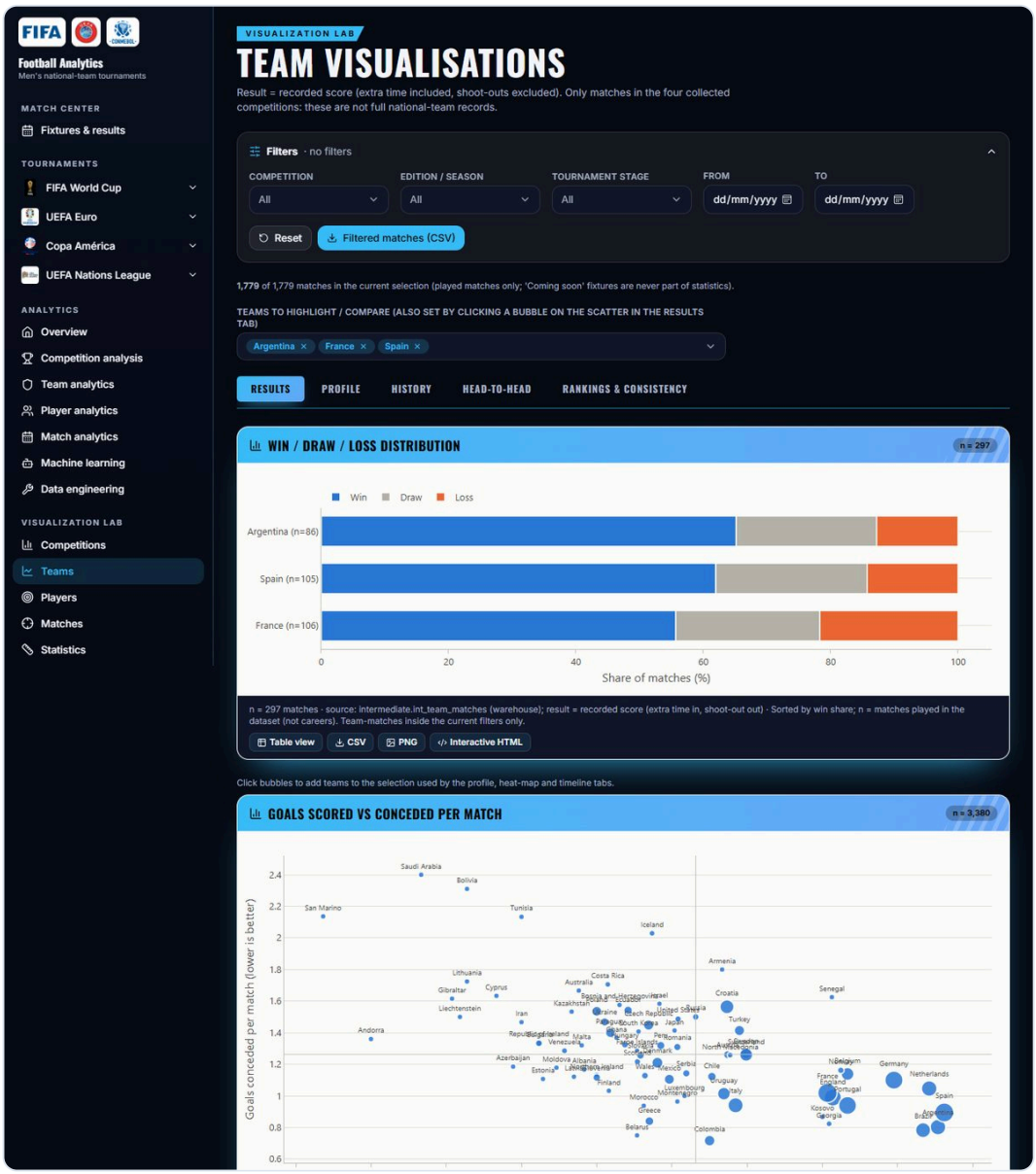
Each edition is themed from its own logo.



Each competition has its own typography and shapes.



World Cup 2030: coming soon, with sourced facts.



Visualization lab: shared filters, click-to-filter, table view and downloads on every chart.

5. Real engineering problems solved

Problem	What I found	Fix
Same match, conflicting sources	Providers disagree on scores, names and dates	Cross-source conflict report, licensed data wins, Wikipedia only fills nulls, raw values kept
Own goals attributed to the wrong team	StatsBomb stores an own goal under the scorer's team; martj42 / Fjelstul under the team that benefits	Normalised per provider and verified against every recorded score (35/35 and 60/60 matches); a "scorers may be incomplete" note when goals do not add up
Scorers missing for whole editions	openfootball goals have event_type "goal" but no goal flag, so the timeline query labelled them "Substitution" (Euro 2000/2004, Copa 2021, World Cup 2026)	Fixed the query; scanned ~840 matches; only 7 matches now lack scorers because no source has them
Data leakage in ML	Features could see the future	Features computed strictly from earlier matches; chronological split; baseline comparison
Upcoming fixtures	Scheduled matches have no result	Kept and shown as "Coming soon"; excluded from every statistic and target
Missing is not zero	Heatmaps counted unplayed matches as missing	Played matches only for completeness metrics; unavailable charts say why
Memory on a free host	API took ~290 MB just to import, ~500 MB with ML	Lazy imports of scikit-learn / statsmodels / mlflow (190 MB at start-up); heavy ML objects pre-computed and shipped as pickles
Slow first request	Scoring the test period and SHAP took ~1 minute each	Pre-computed cache loaded from disk; cold start 4 s in a container
Flags for every country	Historical and variant names	Central name → ISO map with aliases; defunct states and bracket placeholders get a neutral badge, never a wrong flag; a test checks every team in the data
Tabs / charts hidden when data is absent	Commentary and shot data do not exist for most matches	The UI shows only sections the data supports and says what is not recorded

6. Deployment and trade-offs

Browser → Vercel (React, /api proxy) → Cloudflare tunnel → FastAPI on my PC → Postgres (local Docker or Neon)

- Hugging Face Docker Spaces need a paid plan and Render needs a card on file, so the card-free choice was a **tunnel**. The cost is availability: the site is up only while `python tools/serve_public.py` runs, and each tunnel run gets a new address (the script redeploys Vercel automatically).
- A copy of the warehouse (298 MB) is loaded into **Neon**, and a Docker image + `render.yaml` are ready for an always-on host with a one-line change.
- Secrets are never committed; the repository holds code, docs, tests, models, reports and small runtime data only.

7. Interview preparation

The 60-second pitch

> I built an end-to-end football analytics platform. It ingests six open data sources into an immutable Bronze layer, cleans and entity-resolves them in Silver (8,410 canonical players), loads a PostgreSQL star schema, and builds 11 dbt marts with 115 tests. A statistics and ML layer with leakage-free features and a chronological split shows honestly that complex models do not clearly beat an Elo baseline. A FastAPI service serves 85 interactive charts and a Match Center to a React dashboard, where every tournament has its own design built from its logo. It is deployed on Vercel with Neon and a tunnel, and I can explain every trade-off.

Data engineering

Why Bronze / Silver / Gold?

Bronze is immutable and traceable (hash + version), Silver is the cleaned and unified model, Gold is business-ready marts. Each layer can be rebuilt from the one before, and a bug in cleaning never destroys raw data.

How do you merge the same player across sources?

Deterministic rules: same name and birth date, or name containment plus same national team and never in the same line-up. Anything ambiguous goes to a review file. I measured duplicates before and after and kept the raw ids as an xref table.

How do you make the pipeline idempotent?

Steps hash their inputs (and the code for Silver); unchanged inputs are skipped, every attempt is logged in `monitoring.pipeline_execution_logs`, failures retry. Reloading the warehouse drops the dbt views, so the pipeline runs `dbt build` right after.

What data-quality controls exist?

Three layers: Pandera on Silver (13 checks), dbt tests on Gold (115 models and tests), PostgreSQL constraints. Plus a coverage report per edition so gaps are visible rather than hidden.

Tell me about a hard data bug.

Scorers were missing for whole editions. I measured it per season, found one source flagged goals differently (event_type "goal" without the goal flag) so the timeline query mislabelled them, fixed the query, and re-scanned ~840 matches. Own goals were a second trap: two providers store them on opposite teams, so I normalised and verified against every recorded score.

SQL and modelling

Why a star schema?

Facts at match / event / appearance grain with conformed dimensions keep joins simple and fast and make dbt marts easy to build. Surrogate keys decouple the warehouse from source ids.

How do you treat unplayed matches?

They stay in `fact_matches` with status and "Coming soon" detail, but every statistic filters to played matches, and ML targets exclude them.

Machine learning

How did you avoid leakage?

Features use only matches before the match date; the split is chronological (train → 2022, validation → 2024, test after). Nothing from the test period is used for tuning.

Your best model barely beats a baseline. Why report that?

Because it is true and informative: tournament football has high randomness, 55–57 % accuracy on three outcomes is realistic, and bootstrap intervals overlap. I selected on validation log loss, showed calibration, and avoided claiming a win.

Why log loss and not just accuracy?

It scores the probabilities, which matter more than the argmax, and it penalises over-confidence. Accuracy hides that a model is badly calibrated.

How do you explain the model?

Permutation importance and SHAP (global and per outcome), partial dependence, and a note that importance describes the model, not football causality.

Is clustering useful here?

Teams form a continuum (k = 2, weak silhouette and low stability), which I report; player clusters within position groups are more interpretable.

API and front end

Why return Plotly JSON from Python instead of charting in JS?

One implementation for analysis, exports and UI; every availability and "never fabricate" rule lives in one place. The trade-off is server load, handled with caching and pre-computed heavy objects.

How is each tournament themed?

A per-edition design token set (colours extracted from the logo and curated, banner art, pattern, mode) is applied as CSS variables; each competition also has its own typography and corner language. Contrast is checked in tests.

How do you handle flags for 100+ nations?

A central name → ISO map with aliases and the flag-icons library; defunct states and bracket placeholders get a neutral badge, never a wrong flag, and a test verifies every team in the data.

How did you cut API memory?

Lazy imports of scikit-learn, statsmodels and mlflow (290 → 190 MB at start-up), a per-thread reusable database connection, and pre-computed ML caches.

Deployment and operations

Why Vercel plus a tunnel?

Constraints: free only, no card. Vercel hosts the static React app and proxies `/api`; the API needs ~500 MB, Postgres and heavy Python libraries, which no free serverless tier fits. A tunnel keeps everything free but ties availability to my machine; the Docker image and `render.yaml` make an always-on move trivial.

What would you do with more budget?

Run the API on a small always-on container with managed Postgres, add CI (tests + dbt on every push), alerting on freshness, and an autoscaled cache for the ML endpoints.

Behavioural / reflection

What would you improve next?

Event data for the Nations League, a per-match managers source for all editions, assist coverage, richer features (squad value, rest days), and CI/CD.

What are the known limitations?

Assists and managers are missing for some editions, 7 matches lack scorers in every source, Nations League scorers are not linked to player ids, and the live demo depends on one machine.

Numbers worth remembering

1,830 matches · 1,779 played · 51 coming soon	103 teams · 8,410 players
4,494 goals · 1,033,220 events	30 editions · 4 competitions
11 Gold marts · 115 dbt models + tests · 13 Pandera checks	223 Python tests · 6 front-end tests
85 chart ids · 14 dashboard pages	Test period 275 matches · 55–57 % accuracy